R14922040 朱致伶

JPEG Rate Reduction and Quality/Rate Adjustment

1. What the Program Change

The assignment's two tasks are implemented as two modes of one C program:

	Task 1: Huffman optimization	Task 2: Requantization
Main change	Representation of existing symbols	Values of quantized coefficients
Mode	OPTIMIZE	REQUANT
DQT	Unchanged	Replaced
DHT	Recomputed	Can also be recomputed afterward
Image information	Preserved	May lose additional information
Decoded pixels	Identical	May change
Goal	Fewer bits for the same image	Adjust the quality–size tradeoff

JPEG Annex K gives example tables. A standalone JPEG carries the tables needed to decode it; abbreviated streams can receive tables separately. The example tables are not mandatory choices for every image [1].

2. Statistics-based Huffman Rate Reduction

The input JPEG is entropy-decoded into quantized 8x8 DCT blocks. No inverse DCT, RGB conversion, forward DCT or new quantization is performed in optimize.

For sequential JPEG, the encoder gathers frequencies of:

- DC magnitude categories of differences from the preceding DC value of the same component. Predictor state is reset at restart boundaries.
- AC run/size symbols in zigzag order, including end-of-block and runs of sixteen zeros.

These are coded-symbol statistics, not pixel-value frequencies or a histogram of raw signed DCT coefficients. The accompanying coefficient amplitude bits still carry the original information.

Frequent symbols receive shorter codes. JPEG-compatible Huffman construction limits lengths to sixteen bits and reserves the all-ones code point [1]. The encoder then writes new DHT segments and rewrites the entropy-coded stream. Replacing DHT without re-encoding that stream would make the decoder interpret the old bits using incompatible codes.

The C implementation calls jpeg_read_coefficients, copies critical coding parameters, enables optimize_coding, and calls jpeg_write_coefficients and jpeg_finish_compress. libjpeg performs the

statistics pass, table construction, canonical coding, restart handling, edge-block padding, and byte stuffing [2]. This is an application using the libjpeg entropy engine, not a new JPEG codec or a separately hand-written Huffman builder.

The objective concerns **symbol-code lengths**. Actual file size also includes DHT payloads, other markers, padding, byte stuffing, and scan layout. There is no promise that every already-compressed input will get smaller.

3. Quantization and Coefficient Replacement

Let an input coefficient be Cold, with old quantizer step Qold. The original pre-quantization value is unknown. The program uses its reconstruction as an estimate, then requantizes:

$$Fest = Cold * Qold$$

$$Cnew = round(Fest / Qnew)$$

This follows JPEG's scalar quantization/dequantization relationship [1]. The C implementation uses signed 64-bit integer arithmetic and nearest-integer rounding with exact half ties away from zero. It writes the new DQT, replaces coefficients, and recomputes the Huffman tables from the new coded-symbol distribution.

Changing only DQT would alter the dequantized values without compensating the coefficients. For example, an old coefficient 10 with step 8 reconstructs 80; step 16 needs a new coefficient 5 to keep that reconstructed value.

Four quantizer choices are supplied:

Mode	Implementation	Purpose
SCALE	Round and clamp s * Qold	Keep the input matrix's frequency preferences
QUALITY	libjpeg quality preset, 1..100	Familiar encoder control using JPEG example matrix shapes
MPEG4	Round and clamp s * MPEG4_intra_matrix	Compare a different frequency weighti
Table File	Explicit 64/128 quantizer steps	Experiment with user specified luminance/chrominance design

The MPEG-4 intra matrix shape is recorded in FFmpeg's MPEG-4 table source [3]. The program adapts those numbers as JPEG steps. MPEG-4 has different quantization/dequantization rules and DC treatment, so this mode is not a normative MPEG-4 encoder. Both its output format and quantization formula are JPEG.

Larger steps generally lower coefficient precision and rate. They often create more zeros and smaller magnitude categories. This is not strictly monotonic for every image or frequency. With exact step doubling and the selected tie rule, an old coefficient +1 or -1 remains +1 or -1, so the zero count need not rise. Different matrix shapes can change some frequency steps in opposite directions.

Requantizing to finer steps cannot recover discarded information from the original JPEG. It may increase rate without improving fidelity to the unknown uncompressed photograph.

4. Experiment Design

The test photograph is **Matplotlib's bundled grace_hopper.jpg sample [4], 512x600 pixels, YCbCr 4:2:0, single-scan sequential JPEG**. It has no camera EXIF evidence and is not presented as a camera-original file.

The supplied photograph was already encoded with image-specific Huffman tables. Optimizing it gave the same 61,306-byte size, with identical DCT coefficients and pixels. To isolate the effect of the tables, standard rewrote those same coefficients using libjpeg's default Huffman tables. It retained the quantizers, sampling, restart interval and metadata, producing a 62,540-byte control. Verifification confirmed that the source and control have identical quantization and all 462,848 real DCT coefficients.

All following rate-quality measurements use that 62,540-byte control as input. Its decoded pixels match the supplied photographic JPEG, so differences are caused by the measured output transformation.

$$\text{Reduction Rate}(\%) = 100 * (1 - \text{output_bytes}/\text{input_bytes})$$

$$\text{Bits per pixel} = 8 * \text{output_bytes}/(\text{width}*\text{height})$$

$$\text{PSNR} = 10 * \log10(255^2 / \text{RGB_mean_squared_error})$$

PSNR compares output pixels with the decoded input JPEG using the same Pillow decoder. It measures additional distortion; the unknown source photograph's original JPEG distortion is not 3 measured. No visual-quality threshold is assumed, and RGB PSNR alone is not a complete perceptual-quality assessment.

5. Measured Results

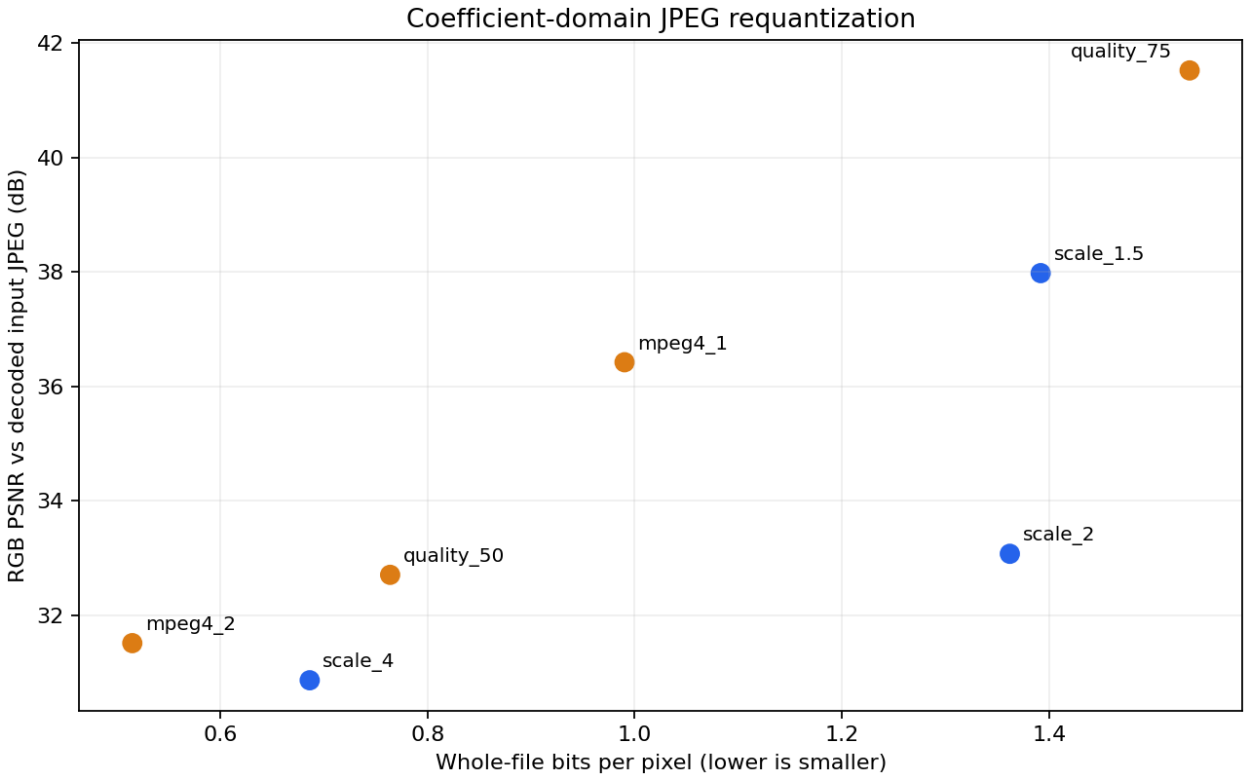
Output setting	Bytes	Reduction	Bits/pixel	RGB PSNR (dB)	Median time (ms)
Default-Huffman control	62,540	0.00%	1.62865	Infinity	—
Optimized Huffman	61,306	1.97%	1.59651	Infinity	6.151
Input DQT x 1.5	53,443	14.55%	1.39174	37.98	8.753
Input DQT x 2	52,300	16.37%	1.36198	33.08	8.264
Input DQT x 4	26,357	57.86%	0.68638	30.87	10.570
JPEG quality 75	58,962	5.72%	1.53547	41.52	8.463
JPEG quality 50	29,337	53.09%	0.76398	32.71	8.434

MPEG-4 shape x1	38,023	39.20%	0.99018	36.42	6.007
MPEG-4 shape x2	19,783	68.37%	0.51518	31.52	6.651

The authoritative exact values are in results/measurements.csv.

Timings are medians of seven warm runs on Linux x86-64 using GCC 13.3.0, -O3, libjpeg-turbo 2.1.5/libjpeg v8 ABI, and Pillow 12.3.0. They include the C program's JPEG reads/writes and temporary file rename, excluding process launch; writes are not fsync'd.

Measured rate and additional distortion Output comparison





6. Findings

1. Huffman coding gives a lossless but limited rate improvement in this assignment.

The default-table control saved 1,234 bytes. Total DHT storage fell from 432 to 188 bytes, including marker/length fields. The remaining net 990-byte saving came from the entropy stream; quantization, image geometry and APP/COM payloads were unchanged. These are whole-file measurements, not an estimate based solely on ideal Huffman bit counts.

2. Quantization offers much larger savings by allowing new distortion.

DQT x4 reduced size by 57.86%, with 30.87 dB additional-distortion PSNR. The MPEG-4-shaped steps at scale 1 gave 39.20% savings at 36.42 dB on the test image. They do not establish that this matrix wins on other photographs or at every matched rate.

3. Matrix choice and rounding matter.

DQT x2 changed 38,802 coefficients, but the zero fraction stayed at about 0.8075 because half ties retained magnitude-one coefficients. MPEG-4 shape x1 changed 72,266 coefficients and increased the zero fraction to about 0.8875. Counting only zeros would miss the size reduction from smaller magnitude categories in the x2 case.

7. Validation

The optimized build passed 28 lossless combinations of grayscale/RGB/CMYK, odd dimensions, 4:4:4/4:2:2/4:2:0 where applicable, progressive/sequential coding, and restart intervals.

For each case, the tests checked coefficient equality, independent decoded-pixel equality, quantization equality and copied APP/COM payloads. Other checks cover quantizer scaling, quality presets, custom matrices, JPEG-valid Huffman code lengths and all-ones exclusion, a negative exact-half requantization tie, repeated optimization, and invalid/truncated input behavior.

The tests also confirm that failed operations preserve an existing destination and leave no temporary output behind.

8. References

[1] ITU-T T.81 / ISO/IEC 10918-1, especially Annex A quantization, Annex B marker syntax, Annex C Huffman specifications, and Annex K example tables: JPEG standard

[2] The libjpeg-turbo Project, public libjpeg API documentation, coefficient access and optimize_coding: libjpeg API documentation

[3] FFmpeg MPEG-4 default intra matrix (ff_mpeg4_default_intra_matrix): FFmpeg MPEG-4 tables

[4] Matplotlib sample photographic image: Matplotlib photographic sample See README.md for build instructions, command examples and supported inputs.